# CSC 3320: System-Level Programming

# Chapter 11: Pointers

Spring 2026

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

# Pointer Variable

- How do variables store in memory?
  - Main memory is divided into bytes.
  - Each byte has a unique address.
  - Each variable occupies one or more memory.
  - The address of the first byte is said to be the address of the variable.

| Address | Contents |
|---|---|
| 0 | 00110011 |
| 1 | 10101010 |
| 2 | 00000111 |
| 3 | 11111000 |
| | … |
| n-1 | 10000100 |

# Pointer Variable

- Example
  - int i = 1; /* suppose sizeof(int) = 4 */
  - Then the address of variable i is 2000

**Can we use a variable to store this address?**

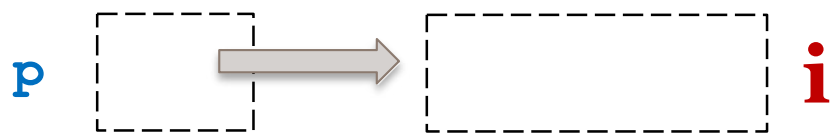
| Address | Contents |
|---|---|
| | .. |
| 2000 | 00000001 |
| 2001 | 00000000 |
| 2002 | 00000000 |
| 2003 | 00000000 |
| | ... |

i

# Pointer Variable

- Pointer Variable
  - Store the address of a variable

| Address | Contents | |
|---|---|---|
| | .. | |
| 2000 | 00000001 | i |
| 2001 | 00000000 | |
| 2002 | 00000000 | |
| 2003 | 00000000 | |
| | ... | |
| | $(2000)_{10}$ | p |




**"p points to variable i"**

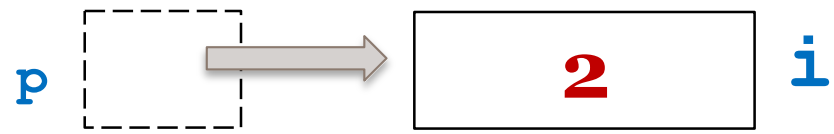
# Declaring Pointer Variables

- Pointer variable must be preceded by an asterisk.
- Every pointer variable points to **objects** of a particular type(*the reference type*).
- Examples
  - **int** * **p**; /* points only to integers */
  - **char** ***str**; /* points only to characters */
  - **double** ***q**; /* points only to doubles */
  - **int i , j , a[10], *p, *q** ; /* p and q point to integers */
- Note: There are no restrictions on what referenced type may be. In fact, a pointer variable can even point to another pointer.

# The Address and Indirect Operators

- **& (address) operator** : get the address of a variable
- *** (indirection) operator** : gain the access to the object that a pointer points to
- Example:

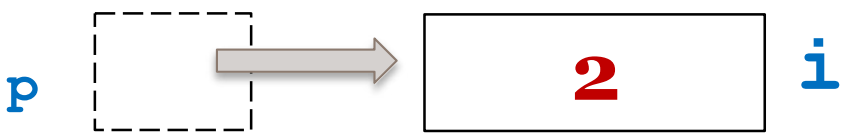
```
int   i=2,  j,  *p;    /* p points nowhere */
p = &i;        /* p  points to integer variable i */
j = *p;  /* same as j=i  */
```



# The Address and Indirect Operators

- **Note:** The address operator can appear in declaration to initialize a pointer
- Example:

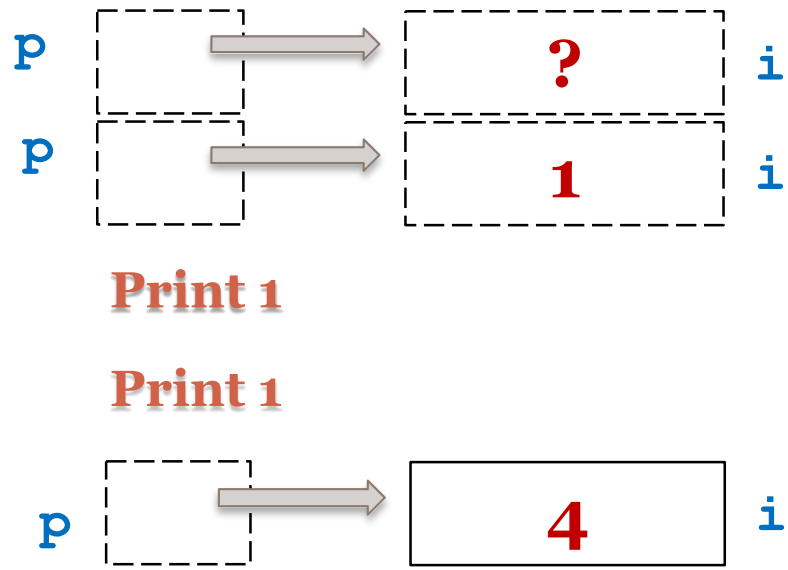
```
int   i=2, *p=&i;   /* p points to variable i */
```



# The Address and Indirect Operators

- **Example**

```
int i, j, *p = &i;
i = 1;

printf("i = %d\n", i);
printf("*p = %d\n", *p);

*p = 4;          Same as i = 4

printf("i = %d\n", i);
printf("*p = %d\n", *p);

j = *&i;         Same as j = i
printf("j = %d\n",j);
```



# The Address and Indirect Operators

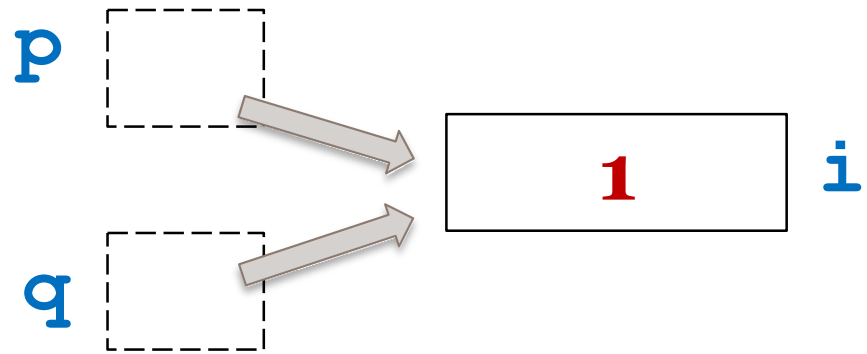
- Guess what will be printed by following statements?
  - int *p;
  - printf("%d", *p);

Note: Never apply the indirection operator to an uninitialized pointer variable.

# Pointer Assignment

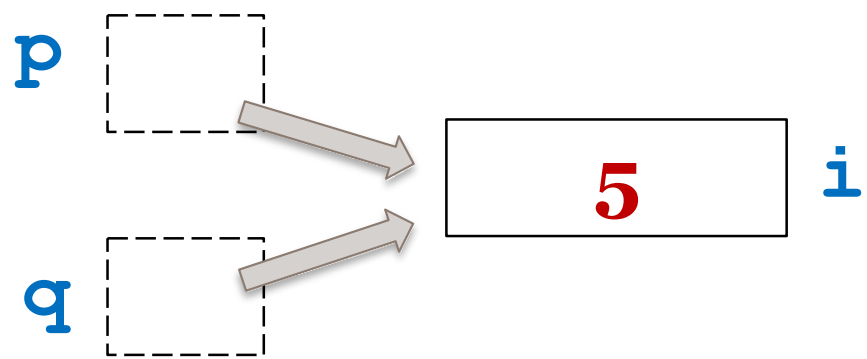
- Example 1

```
int i=1, j=2, *p, *q;
p = &i;
q = p;
```




```
*p = 5;
```

**Same as i = 5**



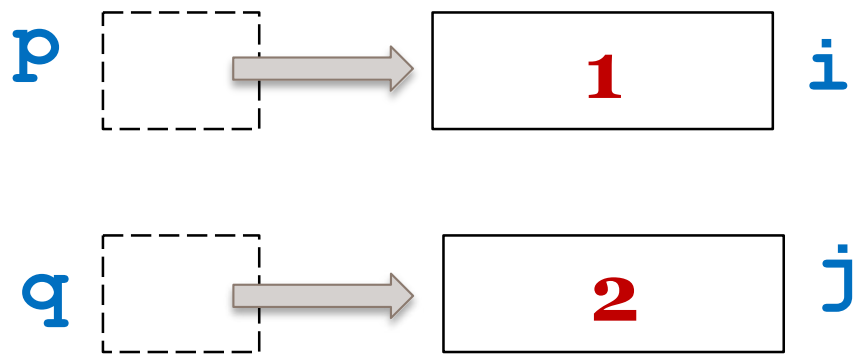
# Pointer Assignment

- Example 2

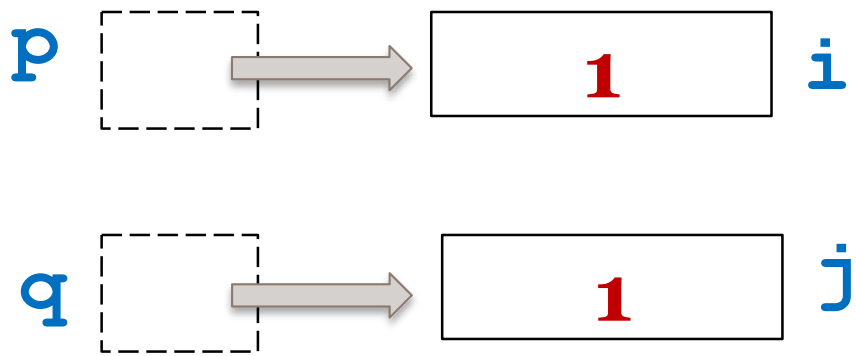
int i=1, j=2, *p, *q;

p = &i;

q = &j;




*q = *p;  Same as j = i



# What is saved in a pointer variable?

```c
#include <stdio.h>

int main()
{   int a, *p;
    p = &a;
    printf("%p\n", &a);
    printf("%p", p);

    return 0;
}
```

Output:
0x7ffdd674ef0c (Address, depends on machine architecture)
0x7ffdd674ef0c

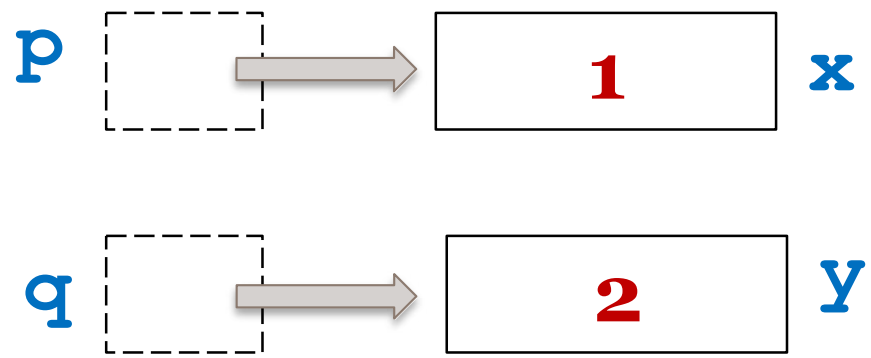
# Pointers as Arguments

```
main(){
    int x = 1, y = 2;
    swap(&x, &y);
    printf("x= %d , y= %d",x,y);
}

void swap(int *p, int *q){
    int temp=0;
    temp = *p;
    *p = *q;
    *q = temp;
}
```

**Output: x= 2, y =1**

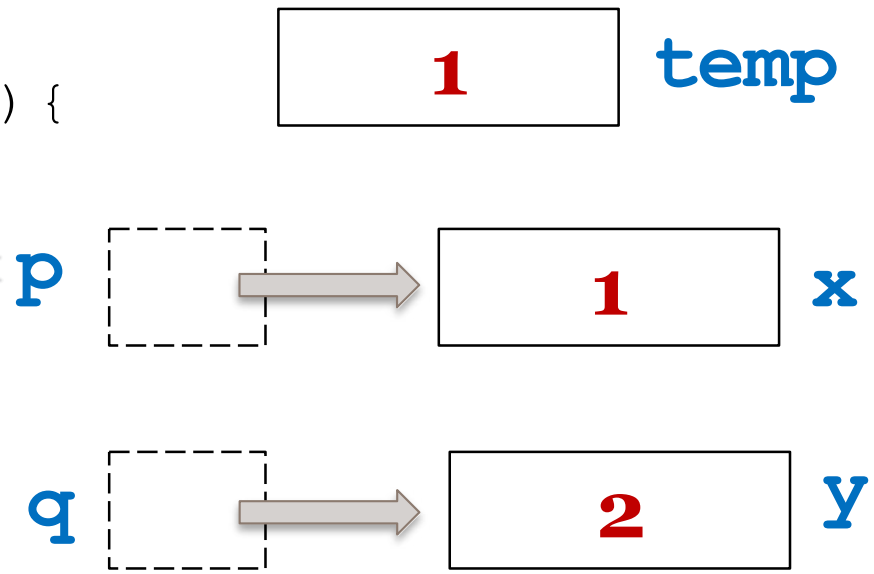
# Pointers as Arguments

```
main(){
  int x= 1,y =2;
  swap(&x,&y);
  printf("x=    %d ,  y= %d",x,y);

}

void swap(int *p, int *q){
    int temp=0;
    temp = *p;
    *p = *q;
    *q = temp;
}
```



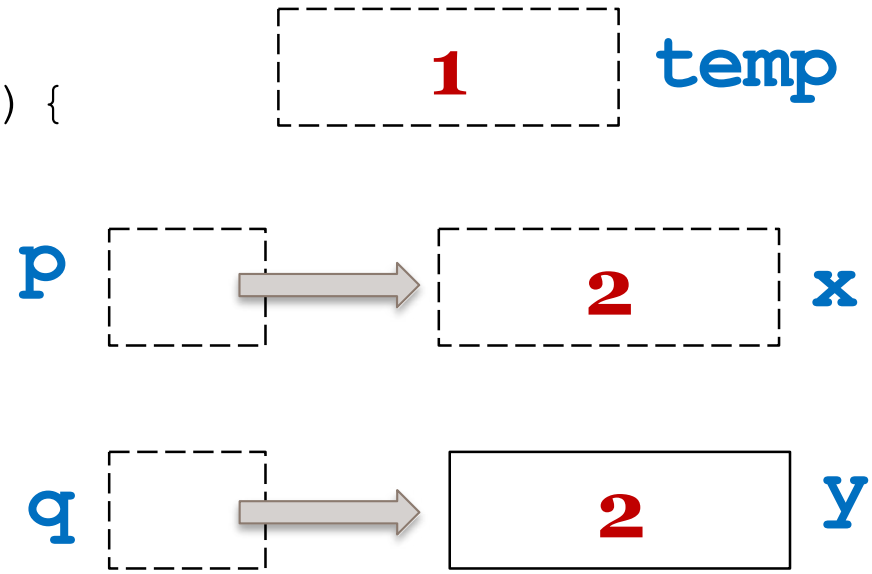
# Pointers as Arguments

```
main(){
  int x = 1, y=2;
  swap(&x,&y);
  printf("x=    %d ,  y=  %d",x,y);
  }

  void swap(int *p, int *q){
      int temp=0;
      temp = *p; Same as temp = x
      *p = *q;
      *q = temp;
  }
```

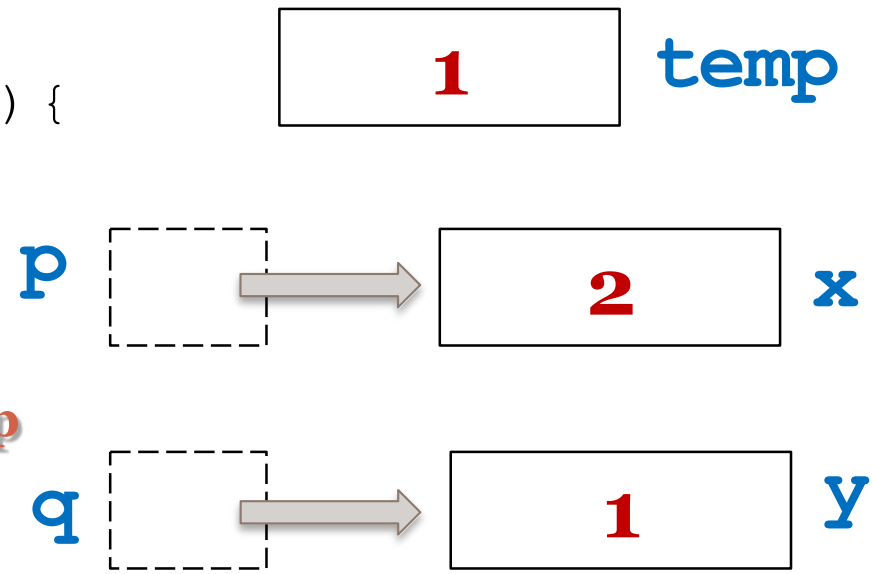


# Pointers as Arguments

```
main(){
  int x  = 1,y= 2;
  swap(&x,  &y);
  printf("x=   %d , y= %d",x,y);
   }

   void swap(int *p, int *q){
       int temp=0;
       temp = *p;
       *p = *q; Same as x= y
       *q = temp;
   }
```



# Pointers as Arguments

```
main(){
  int x = 1, y=2;
  swap(&x,  &y);
  printf("x=   %d ,  y=  %d",x,y);
}

void swap(int *p, int *q){
    int temp=0;
    temp = *p;
    *p = *q;
    *q = temp;
}
```

Same as y= temp